

FIFO Page Replacement Algorithm

Write a C program to implement the FIFO Page Replacement Algorithm. The program should take the number of frames and a reference string (sequence of page requests) as input. For each page in the reference string, determine if it results in a 'Page Hit' or a 'Page Fault'. Display the state of the frames after each request and calculate the total number of page faults and hits.

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string.
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The total number of Page Faults and Page Hits are displayed

Sample Input 1

7

1 3 0 3 5 6 3

3

Sample Output 1

Enter number of pages in reference string:

Enter the reference string:

Enter number of frames:

Total Page Faults: 6

Total Page Hits: 1

How it works

Ref String	Frames	Status
1	1 - -	Fault
3	1 3 -	Fault
0	1 3 0	Fault
3	1 3 0	Hit
5	5 3 0	Fault
6	5 6 0	Fault
3	5 6 3	Fault

Solution:

```
#include <stdio.h>
```

```
int main() {
```

```
    int pages[100], frames[10];
```

```
    int n, f, i, j;
```

```
    int page_faults = 0, page_hits = 0;
```

```
    int index = 0;
```

```
    // Input
```

```
    printf("Enter number of pages in reference string: \n");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the reference string: \n");
```

```
    for(i = 0; i < n; i++) {
```

```
    scanf("%d", &pages[i]);  
}
```

```
printf("Enter number of frames: \n");  
scanf("%d", &f);
```

```
// Initialize frames  
for(i = 0; i < f; i++) {  
    frames[i] = -1;  
}
```

```
// FIFO Algorithm  
for(i = 0; i < n; i++) {  
    int found = 0;
```

```
    // Check HIT  
    for(j = 0; j < f; j++) {  
        if(frames[j] == pages[i]) {  
            found = 1;  
            page_hits++;  
            break;  
        }  
    }  
}
```

```
// If FAULT  
if(!found) {  
    frames[index] = pages[i];  
    index = (index + 1) % f;
```

```
        page_faults++;
    }
}

// Output
printf("\nTotal Page Faults: %d\n", page_faults);
printf("Total Page Hits: %d\n", page_hits);

return 0;
}
```

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string. \
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Sample Input 1

5

1 2 3 4 5

5

Sample Output 1

Enter number of pages in reference string:

Enter the reference string:

Enter number of frames:

Total Page Faults: 5

Total Page Hits: 0

How it works

Step	Page	Frame State	Hit / Fault	Explanation
1	1	1 - - - -	Fault	Frame empty → load 1
2	2	1 2 - - -	Fault	Empty frame → load 2
3	3	1 2 3 - -	Fault	Empty frame → load 3
4	4	1 2 3 4 -	Fault	Empty frame → load 4
5	5	1 2 3 4 5	Fault	Empty frame → load 5

Solution:

```
#include <stdio.h>
```

```
for (int i = 0; i < m; i++) {  
    frames[i] = -1;  
    lastUsed[i] = -1;  
}
```

```
int faults = 0, hits = 0;
```

```
for (int i = 0; i < n; i++) {  
    int page = ref[i];  
    int found = -1;
```

```
    // Check if page is already in frames
```

```
    for (int j = 0; j < m; j++) {  
        if (frames[j] == page) {  
            found = j;  
            break;  
        }  
    }
```

```
}
```

```
if (found != -1) {
```

```
    hits++;
```

```
    lastUsed[found] = i;
```

```
} else {
```

```
    faults++;
```

```
    int replaceIndex = -1;
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (frames[j] == -1) {
```

```
            replaceIndex = j;
```

```
            break;
```

```
        }
```

```
    }
```

```
if (replaceIndex == -1) {
```

```
    int lruIndex = 0;
```

```
    int minTime = lastUsed[0];
```

```
    for (int j = 1; j < m; j++) {
```

```
        if (lastUsed[j] < minTime) {
```

```
            minTime = lastUsed[j];
```

```
            lruIndex = j;
```

```
        }
```

```
    }
```

```
    replaceIndex = lruIndex;
```

```
}
```

```
        frames[replaceIndex] = page;
        lastUsed[replaceIndex] = i;
    }

}

printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);

return 0;
}
```